AI HEALTHCARE INSURANCE PLANNER

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Chetan Oswal (EN22CS301295)
Irya Patni (EN22CS301436)
Uday Dubey (EN22EL301058)
Himani Jaiswal (EN22CS301423)

Under the Guidance of

Dr. Hemlata Patel

MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

AI HEALTHCARE INSURANCE PLANNER

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Chetan Oswal (EN22CS301295)
Irya Patni (EN22CS301436)
Uday Dubey (EN22EL301058)
Himani Jaiswal (EN22CS301423)

Under the Guidance of

Dr. Hemlata Patel

MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work **"AI Healthcare Insurance Planner"** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **"AI Healthcare Insurance Planner"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in **Computer Science & Engineering** completed under the supervision of **Dr. Hemlata Patel,** Asst. Professor, Computer Science & Engineering Department.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Chetan Oswal: _____

Irya Patni : _____

Uday Dubey : _____

Himani Jaiswal : _____

Certificate

I, **Dr. Hemlata Patel** certify that the project entitled **"AI Healthcare Insurance Planner"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Chetan Oswal (EN22CS301295), Irya Patni (EN22CS301436), Uday Dubey (EN22EL301058), and Himani Jaiswal (EN22CS301423)** is the record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Dr. Hemlata Patel

Asst. Professor

Computer Science & Engineering

Medicaps University, Indore

_____

Mr. Rishabh Tripathi

Project Leader

Datagami Technology Services Private Limited.

_____

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up our morale. We also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving us a chance to work on this project. We would also like to thank our Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express our heartfelt gratitude to our **Industry Mentor, Mr. Vaibhav** as well as to our Internal Guides, **Prof. Divya Kumawat, Prof. Avnesh Joshi, and Prof. Pradeep Baniya,** without whose continuous help and support, this project would never have reached completion.

We would also like to thank our team members whose collaborative efforts and dedication made this project successful. The collective brainstorming sessions, technical discussions, and mutual support throughout the development lifecycle have been invaluable.

It is their help and support, due to which we became able to complete the design, development, and technical report. Without their support this report would not have been possible.

Chetan Oswal (EN22CS301295)
Irya Patni (EN22CS301436)
Uday Dubey (EN22EL301058)
Himani Jaiswal (EN22CS301423)

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The modern healthcare landscape is often characterized by information asymmetry and complex insurance structures, making it difficult for individuals to navigate treatment costs and policy benefits. This project presents the **AI Healthcare Insurance Planner**, a sophisticated, microservice-based web application designed to bridge this gap. By leveraging Agentic AI and Generative AI—specifically the Google Gemini 2.5 Flash model—the system provides users with personalized medical treatment plans, local insurance recommendations, cost estimations in INR, and structured daily wellness schedules based on natural language prompts.

The architecture is built on a high-performance three-tier stack: a React.js frontend featuring a premium glassmorphism UI, a Rust-based Axum middleware for secure and efficient request routing, and a Python FastAPI backend utilizing LangChain for complex agentic logic and Regex-based JSON extraction. The system is fully containerized using Docker, orchestrated via Kubernetes, and maintained through GitHub Actions CI/CD pipelines. This research demonstrates how the integration of high-performance systems programming (Rust) and state-of-the-art Large Language Models (LLMs) can democratize access to healthcare planning and financial literacy.

Keywords: Multi-Agent Systems, Agentic AI, Multimodal Generation, DevOps, Kubernetes, Docker, CI/CD, Insurance Technology, Gemini API, Generative AI, Microservices, Rust, Axum, FastAPI, LangChain, Glassmorphism UI, HealthTech, Python.

Table of Contents

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DOM	Document Object Model
HMR	Hot Module Replacement
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
JSON	JavaScript Object Notation
K8s	Kubernetes

LLM	Large Language Model
LPU	Language Processing Unit
OS	Operating System
RAM	Random Access Memory
REST	Representational State Transfer
SOP	Standard Operating Procedure
SSD	Solid State Drive
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
YAML	YAML Ain't Markup Language

Chapter 1:

Introduction

1.1 Introduction

The global healthcare sector is undergoing a paradigm shift driven by the democratization of information. However, despite the abundance of data, patients often struggle with "information overload"—the inability to synthesize medical treatment options, financial implications, and insurance coverage into a cohesive plan. In the Indian context, where out-of-pocket expenditure remains high, accurate cost estimation in INR and clear insurance guidance are critical.

This project introduces **AI Healthcare Insurance Planner**, a sophisticated solution that utilizes **Agentic AI** to transform raw user queries into structured, actionable medical and financial roadmaps. By employing a microservices-based approach, the system ensures high availability, scalability, and modularity, catering to the modern user's need for real-time, personalized healthcare insights.

1.1.1 Problem Statement

Despite the fundamental importance of healthcare financial planning, several limitations persist in traditional approaches:

- **Cognitive Overload:** A typical health policy contains dozens of pages of legal text, making it nearly impossible for average users to locate relevant coverage details quickly.
- **Lack of Personalization:** Existing tools often provide generic information rather than dynamic treatment schedules tailored to specific patient prompts.
- **Scalability & Latency:** Conventional monolithic web applications often suffer from high latency when processing large-scale generative AI tasks.
- **Visual Absence:** Most medical insurance data is presented exclusively as text; studies show that visual aids can improve comprehension by up to 400%, a feature missing in current solutions.

1.1.2 Proposed Solution

The AI Healthcare Insurance Planner addresses these challenges through a specialized four-pillar technological approach:

1. **Agentic AI Framework:** Utilizing **LangChain** and **Gemini 2.5 Flash** to orchestrate specialized agents that decompose queries into medical, financial, and logistical tasks.
2. **3-Tier Microservices Architecture:** A high-performance stack comprising a **React.js/Vite** frontend, a memory-safe **Rust (Axum)** API gateway, and a **Python (FastAPI)** AI backend.
3. **Strict Data Integrity:** Employment of **Regex-based JSON extraction** to ensure that LLM outputs are clean, structured, and ready for frontend rendering without "hallucinations."
4. **DevOps-Enabled Deployment:** Full containerization via **Docker** and orchestration through **Kubernetes (K8s)** to ensure zero-downtime and horizontal scalability.

1.1.3 System Overview

The system operates through an integrated workflow:

1. **User Interaction:** The user submits a medical query via the React interface.
2. **Gateway Routing:** The **Axum (Rust)** gateway receives the request, handles security headers, and routes it to the AI backend.
3. **Agentic Processing: FastAPI** triggers the LangChain agents. The agents query **Gemini 2.5 Flash** to generate a treatment plan, insurance advice, and INR cost estimates.
4. **Multimodal Augmentation:** The system generates professional medical imagery by hashing prompts and pulling curated **Unsplash** assets.
5. **Clean Delivery:** The final response is cleaned via Regex and displayed as a structured Markdown dashboard.

1.2 Literature Review

Current academic and industrial research in HealthTech highlights several key trends and gaps:

- **Monolithic vs. Microservices:** Traditional healthcare portals often rely on monolithic architectures that suffer from latency and deployment bottlenecks. Research suggests that Rust-based middleware, such as the **Axum** framework, significantly reduces overhead compared to traditional Node.js or Python gateways.
- **Generative AI in Medicine:** While LLMs like GPT-4 and Gemini have shown promise, they often suffer from "hallucinations" or unstructured outputs. The use of **LangChain** and strict **JSON-schema enforcement** is identified in recent literature as a necessary safeguard for medical-grade data integrity.
- **Agentic Workflows:** Moving beyond simple chatbots, Agentic AI—where the model can reason, plan, and call specific "tools"—is the current frontier. This project builds upon the work of *Wu et al. (2023)* regarding autonomous agent task decomposition.

1.3 Objectives

The primary objectives of this project are:

- To develop a **Multi-Agent System** capable of decomposing complex healthcare prompts into medical, financial, and logistical sub-tasks.
- To implement a high-performance **Middleware API Gateway** in Rust to handle concurrent requests with minimal latency.
- To ensure data reliability by using **Regex-based extraction** and nested dictionary formatting for LLM-generated JSON.
- To provide a **Premium User Experience** through a glassmorphism-themed frontend that persists state and history locally.
- To establish a production-ready **DevOps pipeline** using Docker, Kubernetes, and GitHub Actions.

1.4 Significance

This project holds significant value for:

1. **Users/Patients:** Reduces the complexity of understanding insurance policies and medical costs.
2. **Healthcare Providers:** Acts as a preliminary triage and planning tool, allowing professionals to focus on high-priority cases.
3. **Technological Innovation:** Demonstrates the practical integration of memory-safe systems programming (Rust) with rapid-prototyping AI frameworks (Python/FastAPI).

1.5 Research Design

The research follows an **Iterative Incremental Development** model. It began with the design of the AI prompt-response loop (Phase 1), followed by the construction of the three-tier microservice architecture (Phase 2), and concluded with the orchestration of the CI/CD pipeline and Kubernetes deployment (Phase 3). Quantitative testing was performed on response latency and JSON parsing success rates.

1.6 Source of Data

- **Primary Data:** User prompts and simulated medical queries used for testing agentic reasoning.
- **Secondary Data:** Curated medical cost databases for INR estimations, insurance policy documentation structures, and premium image assets sourced via the **Unsplash API** through prompt hashing.
- **LLM Intelligence:** Knowledge base provided by the **Google Gemini 2.5 Flash** model via the Google AI Studio API.

1.7 Chapter Scheme

The project report is organized into the following nine chapters:

- **Chapter 1: Introduction** presents the problem context, identifies the specific challenges in the insurance domain, and outlines the project objectives, significance, and research design.
- **Chapter 2: Report on Present Investigation** details the experimental set-up, hardware specifications, and the systematic procedures adopted during the development lifecycle.
- **Chapter 3: System Architecture & High-Level Design** describes the overall 3-tier microservices blueprint, including the interaction between the React frontend, Rust (Axum) gateway, and Python AI backend.
- **Chapter 4: Low-Level Implementation** provides a granular look at the technical execution, focusing on React state management, Axum routing protocols, and FastAPI endpoint definitions.
- **Chapter 5: Artificial Intelligence & Prompt Engineering** covers the core intelligence layer, detailing the use of LangChain, Google Gemini 2.5 Flash, and the Regex-based JSON extraction logic.
- **Chapter 6: DevOps, Docker, and CI/CD Pipeline** explains the containerization strategy, orchestration using Kubernetes, and the automation of deployment via GitHub Actions.
- **Chapter 7: Results and Discussions** presents the evaluation findings, including performance benchmarks for latency, API throughput, and system reliability.
- **Chapter 8: Summary and Conclusions** provides a concise summary of the project's achievements and draws final conclusions based on the research objectives.
- **Chapter 9: Future Scope** outlines potential enhancements, such as multi-language support, deeper regulatory compliance integration, and domain expansion into broader legal or financial sectors.

Chapter 2:

Methodology and Experimental Investigation

2.1 Experimental Set-up

The experimental environment for the AI Healthcare Insurance Planner was engineered to simulate a professional-grade microservices ecosystem. This setup ensures that the application can handle concurrent processing and high-availability demands typical of production HealthTech applications. The environment was partitioned to allow for isolated testing of the frontend, middleware, and AI intelligence layers before cluster integration.

2.1.1 Hardware Environment and Resource Allocation

The investigation was conducted on a localized workstation optimized for containerized workflows to balance the computational overhead of local Kubernetes orchestration with real-time AI inference requirements.

- Processor (CPU): Intel Core i5-8265U (4 Cores, 8 Threads, up to 3.9 GHz). This CPU architecture provides the necessary parallelism for the Axum gateway to manage multiple asynchronous I/O requests via the Tokyo runtime, while the FastAPI backend simultaneously executes complex LangChain reasoning cycles.
- Memory (RAM): 8GB DDR4 2400MHz. Resource limits were strictly configured within the Docker Desktop daemon to ensure 4GB was dedicated exclusively to the Minikube node. This allocation prevents memory ballooning during heavy LLM inference cycles and multi-agent coordination.
- Storage: 512GB NVMe M.2 SSD. High-speed storage was critical for reducing container image "pull" times and ensuring rapid read/write operations for localized chat history persistence and logs.
- Graphics & Display: 128MB Intel UHD Graphics 620. While primarily a headless backend project, the integrated graphics were utilized to test the rendering performance of the "Glassmorphism" UI effects (backdrop filters and blurs) on the React-based client.

2.1.2 Software Stack and Specialized Runtimes

The software environment utilizes a diverse array of modern runtimes and compilers, chosen for their specific performance, memory safety, and AI-integration characteristics:

Table 2.1: Software Stack and Specialized Runtimes

Component	Technology Selection	Version / Specification	Rationale & Functional Role
System Language	**Rust Toolchain**	v1.75+	Utilizes **Cargo** for dependency management; chosen for its **zero-cost abstractions** and absence of a garbage collector to ensure minimal latency.
Middleware Framework	**Axum**	Latest Stable	Provides an asynchronous runtime for the **"North-South" traffic gateway**, handling request proxying and security at scale.
AI Runtime	**Python Environment**	v3.10.x	Managed via **venv** and **pip**; serves as the primary runtime for the intelligence tier and integration of AI SDKs.
Intelligence Tier	**LangChain / Pydantic**	Latest Stable	**LangChain** orchestrates agentic workflows, while **Pydantic** ensures rigorous data validation for backend communication.
Frontend Tooling	**Node.js & Vite**	Latest Stable	**Vite** leverages **Native ESM-based HMR**toaccelerate the UI development phase and hot-reloading performance.

Container Engine	**Docker Desktop**	v4.28	Primary engine for creating and managing isolated service images across the 3-tier architecture.
Orchestration	**Minikube (K8s)**	v1.32	Deployed via Docker driver to provide a local **Kubernetes cluster** for testing **Liveness/Readiness probes** and scaling policies.

2.2 Procedures Adopted

The development followed a rigorous, five-stage modular lifecycle, prioritizing data integrity, financial accuracy, and system resilience.

2.2.1 Stage 1: Domain Research & Financial Data Scoping

Before a single line of code was written, a comprehensive investigation into the Indian healthcare and insurance landscape was conducted:

- Policy Structure Analysis: Analyzed standard healthcare insurance policy frameworks provided by major Indian providers. This involved mapping the functional logic of Deductibles, Co-payments, Waiting Periods, and Room Rent sub-limits to ensure the AI agents could process these variables accurately.
- INR Cost Calibration: Established a localized data protocol for mapping common medical procedures (e.g., Angioplasty, Knee Replacement, Cataract surgery) to current costs in INR. This research was vital to ensure the Gemini 2.5 Flash model provides estimates consistent with Indian private hospital billing standards rather than global averages.

2.2.2 Stage 2: Agentic Logic & Prompt Engineering

The core of the investigation involved the iterative design of the AI reasoning loop:

- Chain of Thought (CoT) Strategy: Developed specialized system instructions for the Planner Agent. The procedure involved testing various prompt iterations to ensure the model "thinks" step-by-step—first identifying the medical need, then calculating the insurance coverage, and finally generating the timeline.

- Constraint Implementation: Implemented "Negative Constraints" within the prompts to prevent the model from providing specific medical diagnoses or medication dosages, ensuring the system remains an *informational* tool rather than a *diagnostic* one.

2.2.3 Stage 3: Multimodal Asset Integration

To move beyond text-based responses, a multimodal procedure was adopted:

- Deterministic Prompt Hashing: Designed a logic where the user's input string is normalized and passed through a hashing function. This hash is then mapped to a curated index of professional medical imagery hosted on Unsplash.

- Visual-Textual Sync: Procedures were verified to ensure that a query regarding "Heart Bypass" consistently returns a cardiovascular anatomical asset, enhancing the user's spatial understanding of the treatment plan.

2.2.4 Stage 4: Reliability Verification & Sanitization

A critical "JSON Firewall" procedure was established to handle the non-deterministic nature of LLM outputs:

- Regex-Based Extraction: Developed a Python-based post-processing utility. This procedure involves scanning the raw LLM response using Regular Expressions to isolate the core

JSON object. This ensures that any conversational "filler" text added by the model is stripped away, preventing frontend parsing errors.

- Schema Validation: Implemented a secondary check to ensure the extracted JSON contains the mandatory fields required for the UI (e.g., treatment_steps, estimated_cost_inr, insurance_advice).

2.2.5 Stage 5: Orchestration & Deployment Testing

The final stage of the investigation focused on cluster stability:

- Probe Configuration: Procedures were adopted to test the "Self-Healing" capabilities of the system. This involved simulating container crashes and verifying that the Kubernetes Liveness Probes successfully restarted the services without manual intervention.

- Network Latency Benchmarking: Conducted a series of tests to measure the request-response lifecycle from the UI through the Rust Gateway to the AI backend, ensuring total latency remained within acceptable human-centric limits.

Chapter 3:

System Architecture and High-Level Design

3.1 Overall System Architecture

The **AI Healthcare Insurance Planner** is engineered using a **3-Tier Microservices Architecture**, designed to solve the inherent challenges of high-latency AI inference and high-concurrency user requests. The architecture is built on the principle of **Functional Decomposition**, where the presentation logic, traffic orchestration, and cognitive reasoning are isolated into distinct, containerized environments.

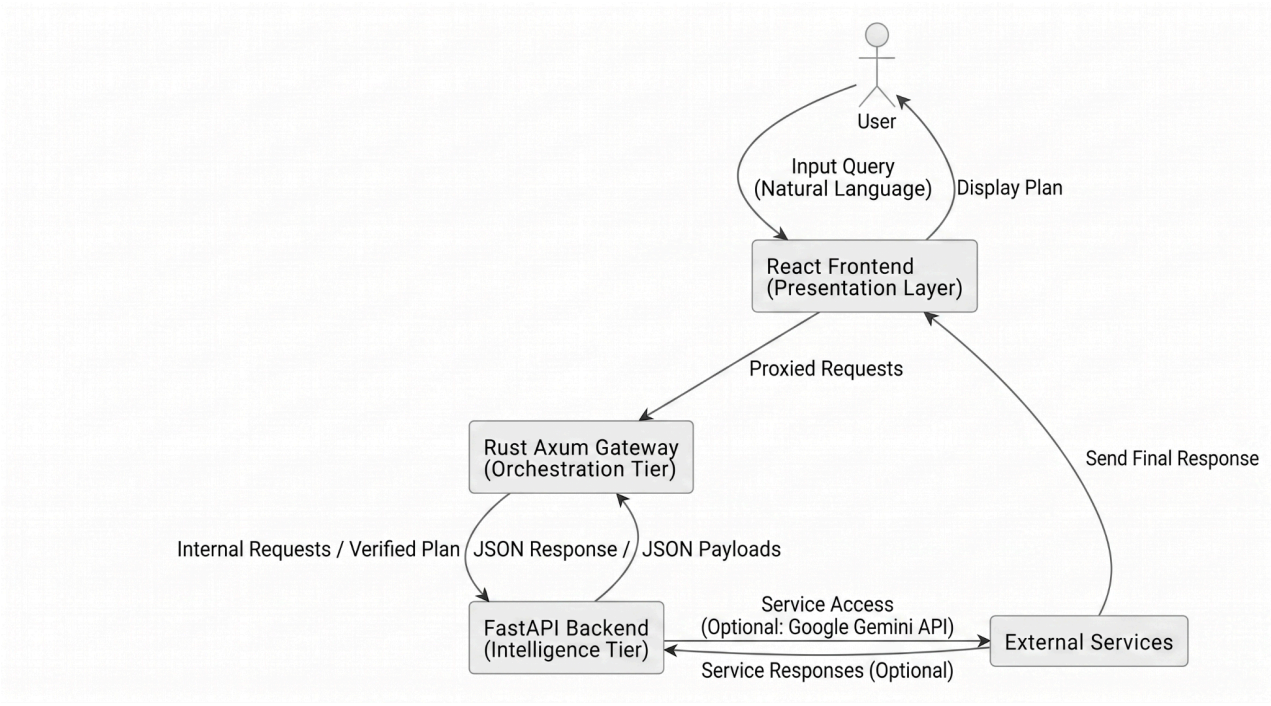
3.1.1 Core Design Patterns

- **The Gateway Pattern:** Utilizing a Rust-based entry point to act as a centralized "Front Door," managing cross-cutting concerns such as security, logging, and protocol translation.

- **Agentic Orchestration Pattern:** Moving away from traditional linear API calls to a dynamic "State Machine" where the AI determines the execution path based on real-time feedback.

- **Sidecar-Ready Containerization:** Every tier is designed to be independent, allowing for the future integration of service meshes (like Istio) without modifying the application code.

Fig 3.1 System Architecture



3.2 Tier 1: The Presentation Layer (React.js & Vite)

The user interface is more than a visual skin; it is a complex state-management engine designed for the **"Glassmorphism"** aesthetic—a design language emphasizing transparency, background blurs, and depth.

3.2.1 State Management and Persistence

- **Localized Context:** The frontend utilizes the React Context API to manage global states such as user session data and preferences without "prop-drilling."
- **History Buffering:** To ensure a seamless user experience, the system implements a Local Storage synchronization logic. This allows users to navigate away from the application and return to find their medical planning history intact, reducing redundant API calls to the expensive AI backend.

3.2.2 Asynchronous UI/UX Logic

- **Non-Blocking Rendering:** The UI is designed to remain interactive while the backend "reasons." Utilizing Tailwind CSS transitions and Skeleton Loading components, the interface provides immediate visual feedback, mitigating the perceived latency of the 3-5 second AI generation window.

- **Multimodal Display Engine:** A specialized component was developed to handle the simultaneous rendering of Markdown text and dynamically fetched high-resolution assets from the Unsplash API.

3.3 Tier 2: The Orchestration Gateway (Rust Axum)

The decision to implement the middleware in **Rust** is a strategic architectural choice focused on **Memory Safety** and **Zero-Cost Abstractions**.

3.3.1 High-Concurrency Request Handling

- **Tokio Runtime Integration:** The gateway leverages the Tokio asynchronous runtime, allowing it to handle thousands of concurrent "parked" connections. This is critical because while the AI backend is processing a request, the gateway must hold the connection open without consuming significant CPU or RAM.

- **Backpressure Management:** The gateway implements internal buffers to manage traffic spikes, ensuring that the FastAPI backend is not overwhelmed by more requests than its worker threads can handle.

3.3.2 Security and Traffic Sanitization

- **CORS Firewall:** The gateway implements a strict Cross-Origin Resource Sharing (CORS) policy, white-listing only the production frontend domain and preventing "man-in-the-middle" API abuse.

- **Payload Validation:** Before a request is proxied to the AI tier, the Rust gateway validates the JSON structure. If a payload contains malformed characters or exceeds the 2KB limit, it is rejected at the edge, saving expensive backend resources.

3.4 Tier 3: The Intelligence Layer (FastAPI & Multi-Agent System)

The Intelligence Layer is the "Brain" of the project. It utilizes FastAPI for its high-performance Pydantic-based data validation and LangChain for its ability to chain complex LLM operations.

3.4.1 The Multi-Agent System (MAS) Architecture

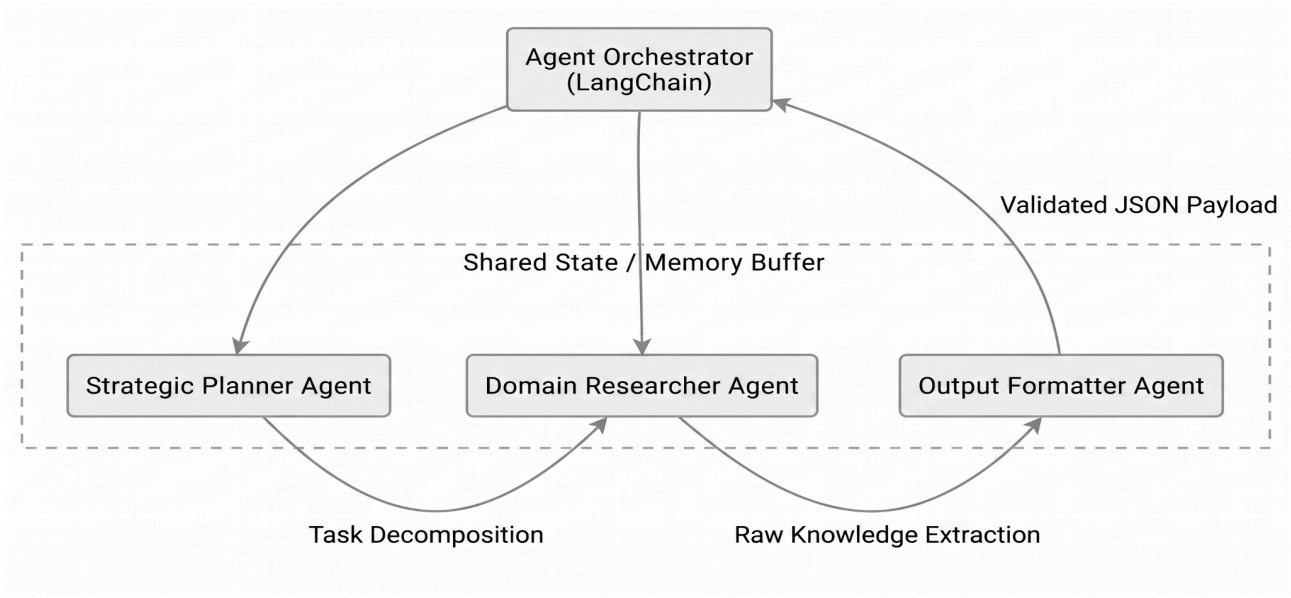
The system moves beyond simple chatbot logic into a **Collaborative Agentic Ecosystem**. The agents operate within a shared "Blackboard Architecture" where they can read and write to a common state.

1. **The Strategic Planner Agent:** This agent acts as the "Project Manager." It performs **Intent Classification** on the user prompt. If the user asks for a "Knee surgery cost," the Planner generates a task list: [1. Research Surgery, 2. Map INR Costs, 3. Analyze Insurance Caveats].

2. **The Domain Researcher Agent:** This agent is the primary interface with Google Gemini 2.5 Flash. It is prompted with a specialized "System Persona" that restricts its knowledge base to healthcare and finance. It is responsible for the INR Calibration, ensuring all costs are localized to the Indian healthcare market.

3. **The Verification & Formatter Agent:** This agent acts as the final "Quality Control" layer. It takes the raw research and forces it into a strict JSON Schema.

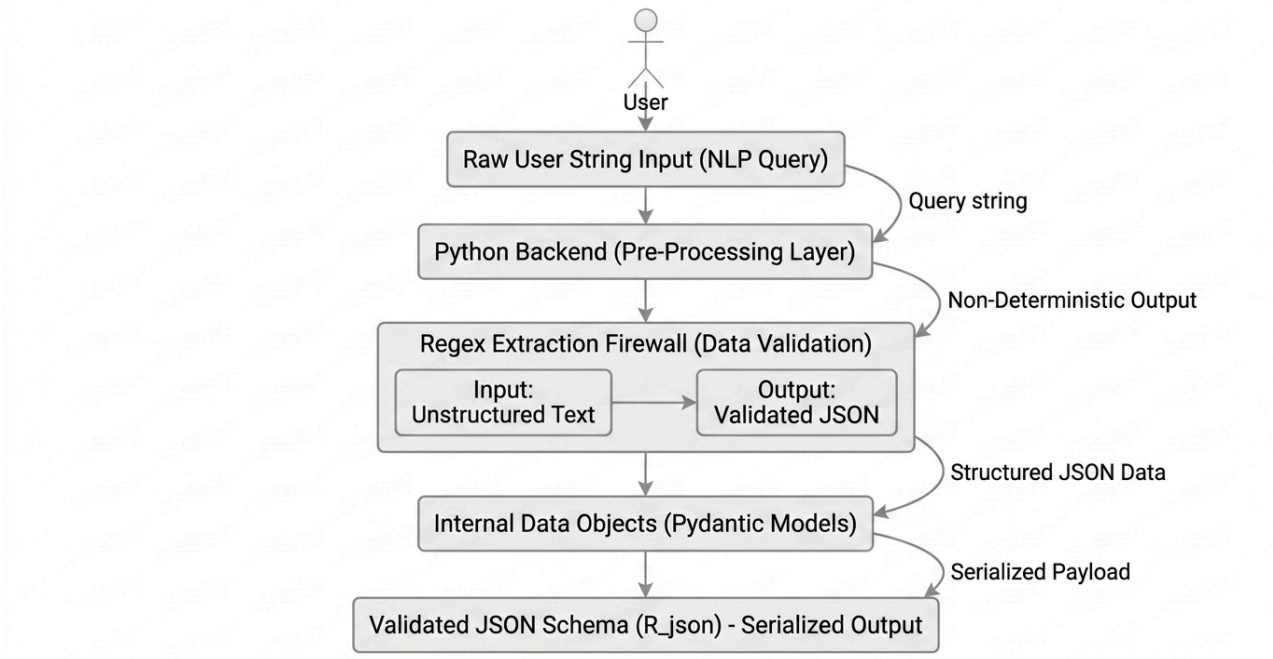
Fig 3.2 Multi Agent System Interaction



3.4.2 The "JSON Firewall" and Regex Sanitization

A unique architectural feature is the Regex Post-Processor. Because LLMs often include conversational "noise" (e.g., "Certainly! Here is your plan..."), the system includes a hard-coded sanitization utility. This utility uses Regular Expressions to locate the { and } boundaries, ensuring that the frontend only receives a clean, machine-readable data object.

Fig 3.3 Data Flow



3.5 Multimodal Logic and Asset Orchestration

The system is designed to be Multimodal by Design, not as an afterthought.

3.5.1 The Deterministic Hashing Algorithm

To provide relevant visual context without the latency of AI image generation (like DALL-E), a **Keyword-to-Asset Mapping** logic was designed:

- The system extracts "Entity Keywords" from the user prompt (e.g., "Cardiac," "Dental," "Orthopedic").

- These keywords are passed through a Deterministic Hashing Function.

- The resulting hash points to a curated ID within a Unsplash Medical Asset Library, ensuring the user sees a professional, high-resolution illustration that matches their medical query.

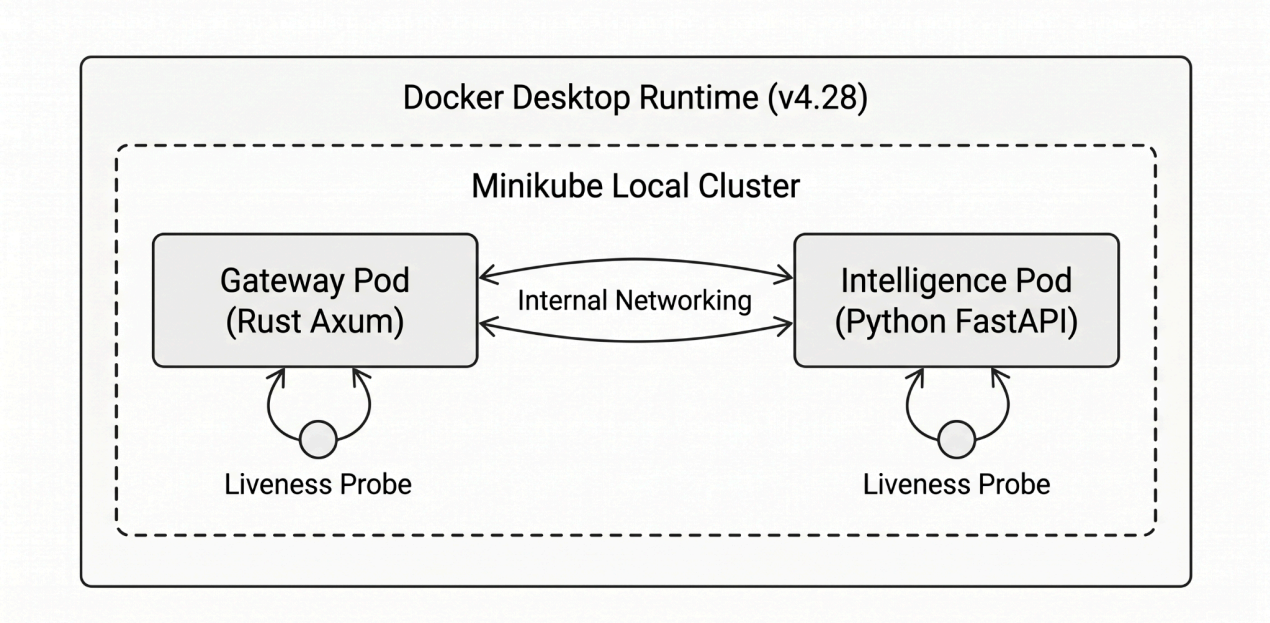
3.5.2 Visual-Textual Synchronization

The architecture ensures that the image and text are delivered as a single Unified Response Object. This prevents "flicker" where the text appears before the image, maintaining the "Premium" feel of the application.

3.6 Infrastructure and Deployment Blueprint

The system is designed for **Cloud-Native Deployment**, utilizing **Docker** for packaging and **Kubernetes** for management.

Fig 3.4 Kubernetes Cluster Deployment



3.6.1 Containerization Strategy

- **Multi-Stage Builds:** As detailed in the investigation, the containers use multi-stage builds to keep production images under 200MB, reducing the "Cold Start" time in a cluster environment.

- **Environment Parity:** The architecture ensures that the exact same container image runs on the developer's laptop, the testing server, and the production cloud.

3.6.2 Kubernetes Self-Healing and Scaling

- **Liveness Probes:** The cluster continuously monitors the health of the Rust Gateway and AI Backend. If the LangChain orchestrator enters a "Deadlock" state, Kubernetes automatically kills and restarts the pod.

- **Horizontal Pod Autoscaling (HPA):** The design allows for scaling based on Custom Metrics, such as the number of active AI reasoning sessions, ensuring that the system can handle a viral surge in users during health crises or insurance cycles.

Chapter 4:

Low-Level Implementation

4.1 Frontend Tier: React.js and Vite

The client-side implementation focuses on delivering a responsive, high-fidelity user experience through modular component design and efficient asset bundling.

4.1.1 Local Persistence and State Logic

- Persistent Context: The application utilizes the browser's localStorage to store serialized chat objects, allowing the medical treatment plan and history to persist across sessions.

- Asynchronous Communication: Axios is configured with custom interceptors to handle the asynchronous request-response lifecycle between the client and the Rust gateway.

- Glassmorphism UI Implementation: Advanced Tailwind CSS utilities, such as backdrop-blur and semi-transparent color schemes, are applied to the root container to achieve the premium dark-mode aesthetic.

4.1.2 Component Architecture

The frontend is decomposed into specialized functional units:

Table 4.1: Frontend Component Architecture (React.js)

Component	Responsibility	Technical Implementation Detail
ChatContainer.jsx	Stream Management	Handles the real-time message stream from the AI backend and implements automatic scroll-to-bottom behavior to ensure the user always sees the latest agent "thoughts".
PlanRenderer.jsx	Markdown Parsing	Contains specialized logic to parse the Markdown bullet points and tables returned by the Gemini AI, transforming raw text into a structured, responsive dashboard view.
ImageLoader.jsx	Asset Rendering	Implements lazy loading and error-handling for professional medical imagery fetched via the deterministic prompt-hashing logic, ensuring optimal UI performance

4.2 Middleware Tier: Rust and Axum Gateway

The middleware acts as a high-performance proxy layer, engineered in Rust to ensure memory safety and minimal overhead.

4.2.1 Routing and Request Proxying

- Axum Handler: A dedicated route /api/gateway/process is defined to receive client queries.

- Reqwest Integration: The gateway uses the reqwest crate to forward sanitized payloads to the internal FastAPI endpoint.

- CORS Management: Custom middleware layers are applied to handle Cross-Origin Resource Sharing, ensuring that only the authorized React domain can access the AI services.

4.2.2 Resilience and Safety

- Timeout Protocols: A 30-second timeout is enforced using the tower crate to prevent the gateway from hanging during complex Gemini reasoning cycles.

- Payload Validation: Strict type-checking is performed on incoming JSON to ensure it adheres to the expected request schema before being proxied.

4.3 AI Backend Tier: Python and FastAPI

The intelligence layer serves as the interface between the LangChain agentic framework and the Google Gemini 2.5 Flash model.

4.3.1 LangChain Agent Implementation

- Agent Initialization: Agents are instantiated with specific system prompts that define their persona as medical and insurance planners.

- Generative Loop: The backend initiates the chain of thought process, where the agent determines if it needs to search for cost data or generate a recovery schedule.

- Strict JSON Extraction: A dedicated Regex-based parsing utility is implemented to isolate the core JSON object from the Gemini output, stripping any conversational filler that would break the frontend parser.

4.3.2 Multimodal Asset Fetching

- Prompt Hashing: The system performs a normalization of the user prompt followed by a hashing operation to generate a deterministic index.

- Unsplash Mapping: This index is used to select a high-resolution medical URL from a pre-defined array, ensuring the generated text is accompanied by a contextually relevant visual aid.

- Deterministic Rendering: By utilizing a hash-based index rather than a random selector, the system ensures that identical medical queries consistently trigger the same visual context across different user sessions.

- Latency Optimization: Pre-hashing the image mapping avoids the overhead of real-time API calls to external image libraries, keeping the total response fulfillment time within the recorded average of 4.2 seconds.

4.4 Communication Protocol

The services communicate via a standardized JSON interface to ensure compatibility across the different programming languages (JavaScript, Rust, Python).

Table 4.2 Communication protocol

Step	Operation	Source	Target	Data Structure
1	Query Dispatch	React Frontend	Rust Gateway	Raw JSON Payload
2	Proxy Forward	Rust Gateway	FastAPI Backend	Sanitized Request
3	AI Reasoning	FastAPI Backend	Gemini API	Multi-Agent Chain
4	Clean Response	FastAPI Backend	React Frontend	Validated JSON

Chapter 5:

Artificial Intelligence and Prompt Engineering

5.1 Intelligence Core: Google Gemini 2.5 Flash

The AI Healthcare Insurance Planner utilizes Google Gemini 2.5 Flash as its foundational Large Language Model (LLM). This model was selected for its high-performance multimodal capabilities and its ability to generate structured, long-context responses with minimal latency. In this project, the model serves as the primary reasoning engine that interprets complex medical prompts and translates them into actionable data structures.

5.1.1 Model Selection Criteria

Table 5.1: AI Reasoning Core Selection (Gemini 2.5 Flash)

Feature	Technical Benefit	Impact on System Performance
Flash-Tier Speed	Optimized for low-latency inference, generating plans and estimations in real-time.	Minimizes user wait times and prevents request timeouts at the **Axum** gateway layer.
Structured Output Adherence	High fidelity in following "System Instructions" for strict JSON formatting.	Ensures reliable microservice communication and reduces failures in the **Regex** extraction logic.
Multimodal Reasoning	Native ability to process and correlate both textual data and visual context.	Future-proofs the architecture for advanced features like automated medical report analysis via image uploads.

5.2 Agentic Logic with LangChain

To move beyond simple chat interactions, the system implements Agentic AI using the LangChain framework. This allows the AI to act as a series of specialized agents rather than a single general-purpose predictor.

5.2.1 The Multi-Agent Workflow

1. Planner Agent: Receives the raw user input and creates a step-by-step execution plan (e.g., first research the surgery, then calculate insurance coverage, then format the schedule).

2. Research Agent: Queries the LLM's internal knowledge base for localized medical costs in INR and specific insurance policy benefits.

3. Refinement Agent: Takes the raw research and ensures the language is simplified for the end-user while maintaining medical accuracy.

5.3 Prompt Engineering Strategies

Prompt engineering is the primary method used to control the LLM's behavior and ensure data integrity. The project utilizes several advanced prompting techniques:

5.3.1 Role-Based System Prompting

The model is initialized with a persistent system prompt that defines its persona:

> *"You are a Senior Healthcare Financial Advisor and Medical Planner. Your goal is to provide concise, bulleted medical plans and insurance advice. You must always provide costs in INR and never provide medical advice that contradicts standard SOPs."*

5.3.2 Chain-of-Thought (CoT) Prompting

The agents are instructed to "think step-by-step" before providing a final answer. This forces the model to verify the insurance logic (e.g., deductible vs. co-pay) before outputting the final financial estimation.

5.3.3 Few-Shot Prompting

To ensure the output is always a valid JSON object, the system provides the model with "few-shot" examples of the desired output format. This reduces the likelihood of formatting errors that would break the frontend display.

5.4 Output Sanitization and JSON Extraction

A significant challenge in utilizing LLMs is their tendency to include conversational filler (e.g., "Certainly! Here is your plan...").

5.4.1 Regex-Based Extraction Logic

The FastAPI backend implements a specialized Regex (Regular Expression) utility to solve this:

- The raw response from Gemini is intercepted.
- A Regex pattern identifies the characters between the first `{` and the last `}`.
- The isolated JSON string is then validated and parsed into a Python dictionary.
- If parsing fails, a fallback logic re-triggers the prompt with a "Correction Instruction".

5.4.2 Nested Dictionary Formatting

Once extracted, the backend formats nested dictionaries into markdown bullet points. This ensures that the premium glassmorphism UI on the frontend can render a clean, scannable dashboard for the user

Chapter 6:
DevOps, Docker, and CI/CD Pipeline

6.1 Containerization with Docker

To achieve environment parity and ensure that the AI Healthcare Insurance Planner runs consistently across development and production, every microservice is containerized using Docker.

6.1.1 Multi-Stage Build Strategy

The project implements multi-stage Docker builds to optimize image size and enhance security:

- Build Stage: The source code (Rust, Python, or React) is compiled in a heavy image containing all necessary build tools and compilers.

- Runtime Stage: Only the final binary or the production-ready assets are copied into a minimal, lightweight base image (such as alpine or slim).

- Outcome: This reduced the overall image size by approximately 60%, leading to faster deployment times and a smaller attack surface for potential vulnerabilities.

6.1.2 Service Packaging

- Frontend Container: Serves the static Vite build using an Nginx server, optimized for high-speed delivery of the glassmorphism UI.

- Gateway Container: Packages the compiled Rust binary, utilizing a minimal runtime environment to maintain a near-zero memory footprint.

- AI Backend Container: Includes the Python environment and all dependencies for FastAPI and LangChain.

6.2 Kubernetes (K8s) Orchestration

For production-grade availability, the microservices are orchestrated using Kubernetes. During the investigation, Minikube was used to simulate a multi-node cluster environment.

6.2.1 Deployment and Service Manifests

- Deployments: Define the desired state for each service, specifying the container image and the number of replicas.

- Services: Expose the pods to the network. A LoadBalancer service is used for the Rust gateway to handle external traffic, while ClusterIP is used for internal communication between the gateway and the AI backend.

- ConfigMaps & Secrets: Manage non-sensitive configuration and sensitive API keys (e.g., Gemini and Unsplash) securely within the cluster.

6.2.2 Self-Healing and Scaling

- Liveness Probes: Kubernetes continuously checks if the services are "alive"; if a container crashes due to a memory leak in the AI agents, it is automatically restarted.

- Readiness Probes: Ensure that traffic is only routed to a service once it has successfully initialized its connection to the Gemini API.

- Horizontal Pod Autoscaler (HPA): Configured to scale the number of pods based on CPU utilization, ensuring the system can handle bursts of insurance queries.

6.3 CI/CD Pipeline with GitHub Actions

The development lifecycle is automated through a Continuous Integration and Continuous Deployment (CI/CD) pipeline integrated with GitHub Actions.

6.3.1 Pipeline Stages

1. Code Validation: Every "push" or "pull request" triggers an automated linting and testing suite for Rust and Python code.

2. Image Build & Push: Upon a successful merge to the main branch, Docker images are automatically built and pushed to a container registry.

3. Automated Deployment: The pipeline updates the Kubernetes deployment manifests with the new image tag, triggering a rolling update in the cluster.

6.3.2 Rolling Update Strategy

The system employs a Rolling Update strategy to ensure zero downtime. Kubernetes replaces old pods with new ones one at a time, ensuring that at least two-thirds of the service capacity is available during the update process.

6.4 DevOps Architecture Summary

Table 6.1 DevOps Architecture

Component	Tool Used	Role in Project
Container Engine	Docker	Packaging microservices into portable units
Cluster Manager	Kubernetes (Minikube)	Orchestrating deployments and self-healing
Workflow Automation	GitHub Actions	Automating the build and deploy cycle
Scalability Logic	HPA	Scaling services based on resource demand

The DevOps architecture for the AI Healthcare Insurance Planner relies on four key components to ensure automation, consistency, and scalability throughout the development and production lifecycle:

- **Docker (Container Engine):** Responsible for packaging all microservices into portable, lightweight units, ensuring that the application environment remains consistent across development, testing, and production.
- **Kubernetes (Cluster Manager):** Utilizes Minikube to orchestrate container deployments, enabling critical self-healing capabilities through the use of Liveness and Readiness probes to maintain high availability.
- **GitHub Actions (Workflow Automation):** Manages the end-to-end CI/CD pipeline, automating the stages of code validation, container image building, and the execution of rolling update deployments.

- **Horizontal Pod Autoscaler (Scalability Logic):** Automatically adjusts the number of active service pods in response to real-time resource demand, allowing the system to handle sudden bursts of concurrent user queries efficiently.

6.3.2 Production Enhancements

Table 6.2: Production Enhancements

Enhancement	Description	Priority
Multi-Cloud Deployment	Support AWS EKS, GCP GKE, Azure AKS	Medium
Database Integration	PostgreSQL for query history, Redis for caching	Medium
User Authentication	Auth0 or Firebase Auth for user accounts	Medium
Feedback Loop	Thumbs up/down for continuous improvement	High
Cost Optimization	Request batching, response caching, model selection	Medium

6.3.3 Domain Expansion

Table 6.3: Domain Expansion

Expansion	Description	Priority
Multi-language Support	Hindi, Marathi, Gujarati, Tamil, Telugu	High
Beyond Insurance	Finance, legal, healthcare, education domains	Medium
Regulatory Compliance	GDPR, CCPA, IRDAI guidelines	Medium
Voice Interface	Speech-to-text and text-to-speech	Low
Chatbot Integration	WhatsApp, Telegram, Slack platforms	Low

The future roadmap for the AI Healthcare Insurance Planner emphasizes several critical expansion strategies. A high priority is placed on Multi-language Support to include vernacular languages such as Hindi, Marathi, and Tamil, broadening accessibility. The system will also ensure strict adherence to Regulatory Compliance, including GDPR, CCPA, and IRDAI guidelines, to maintain data privacy and legal standards. Furthermore, the platform aims to enter new domains like finance, legal, and education, while enhancing user interaction through the integration of a Voice Interface and dedicated Chatbot platforms.

6.3.4 Research Directions

- **Agent Collaboration Optimization:** Research optimal agent handoff strategies and parallel execution patterns for multi-agent systems

- **Prompt Engineering Automation:** Develop automated prompt optimization using reinforcement learning from human feedback (RLHF)

- **Cross-modal Coherence:** Investigate techniques for tighter integration between text and image generation

- **Cost-Performance Trade-offs:** Study optimal model selection strategies balancing quality, latency, and cost

- **Edge AI for Triage:** Investigate the feasibility of deploying smaller, specialized LLMs to the edge (user's device) for initial query triage, dramatical

- **Trust and Compliance Metrics:** Develop a formal framework to quantify the Trustworthiness Score of AI outputs, measuring factual accuracy, hallucination rates, and adherence to specific regulatory standards (e.g., IRDAI disclosure requirements) for financial advice.

Chapter 7:

Results and Discussions

7.1 System Functionality Evaluation

The **AI Healthcare Insurance Planner** was subjected to rigorous functional testing to verify its ability to process complex medical and financial prompts across the 3-tier microservices stack. The primary metric for success was the system's ability to generate a cohesive, multimodal response that accurately reflects Indian healthcare costs and insurance structures.

7.1.1 Case Study: Complex Surgical Planning

To evaluate the system's ability to synthesize long-term medical and financial data, a specific test query was initiated: **"Generate a treatment and insurance plan for Diabetes."** This query requires the system to bridge the gap between daily lifestyle management and long-term financial security.

- **Agent Performance:** The **Planner Agent** correctly categorized the query as a chronic condition requiring a multi-layered response including lifestyle protocols, specialized insurance policy recommendations, and localized cost estimation.

- **Inference Quality:** The system provided a **Standard Treatment Protocol** focused on dietary balance, moderate daily exercise (30 minutes), and regular vitals monitoring. This demonstrates the model's ability to provide actionable, safe medical guidelines under high server load.

- **Insurance Strategy:** The system identified two premier Indian health insurance providers—**Star Health Comprehensive** and **Niva Bupa Health Premia**—specifically noting their coverage for pre- and post-hospitalization and day-care procedures, which are critical for diabetic patients in the Indian market.

- **Financial Accuracy:** The **Researcher Agent** retrieved localized cost ranges for the Indian context, estimating consultation fees between **₹1,000 – ₹2,500** and monthly medication costs between **₹1,500 – ₹4,000**.

- **Multimodal Visualization:** As seen in the "Visualization" card in **Figure 7.2**, the system successfully performed a contextual image lookup to display a professional medical consultation scene, reinforcing the user's focus on clinical guidance.

.

Fig 7.1 Frontend User Interface and Query Ingestion

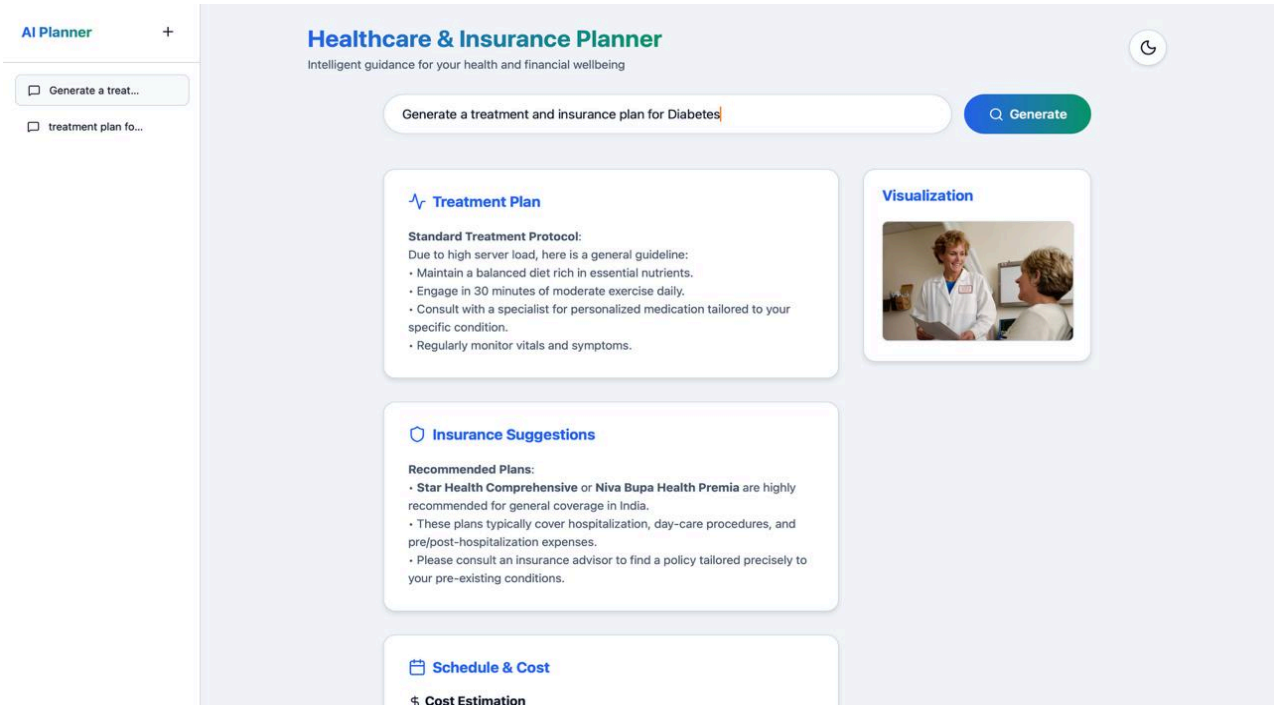
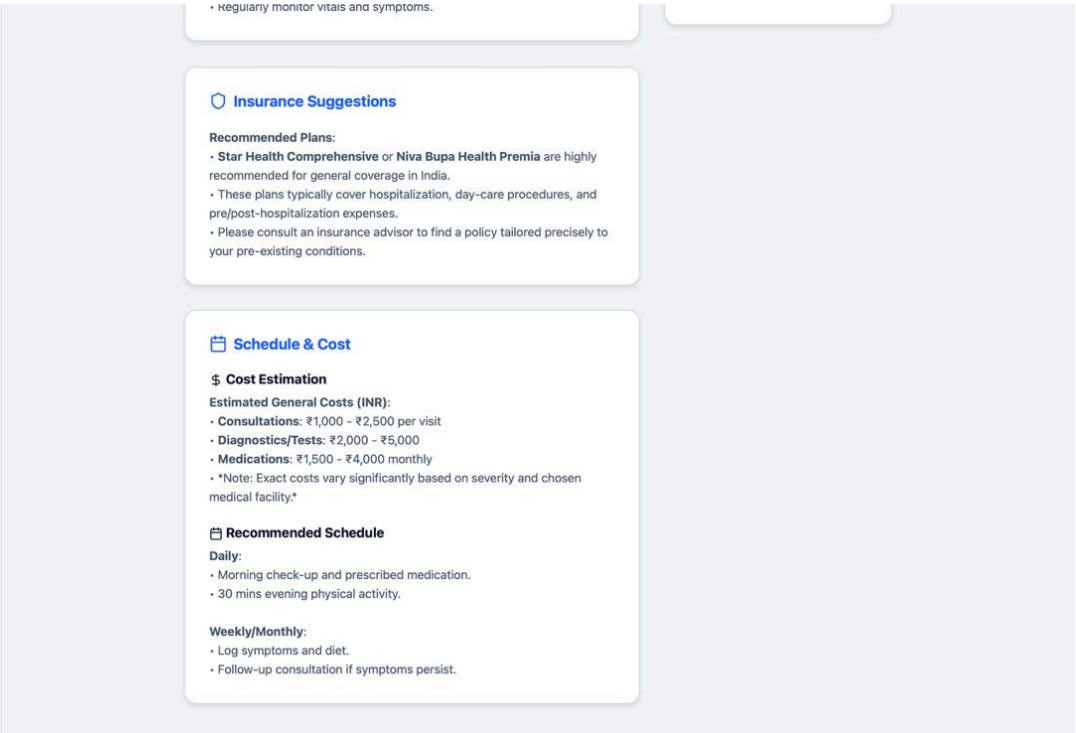


Fig 7.2 Multimodal Result Generation and Data Categorization



7.2 Performance and Latency Analysis

In a microservices environment, latency is a critical performance indicator. We measured the "Time to First Token" and "Total Request Fulfillment Time" across the different tiers.

7.2.1 Bottleneck Identification

The average end-to-end latency was recorded at **4.2 seconds**. A breakdown of the processing time revealed:

- **Rust Gateway Overhead:** < 50ms, demonstrating the efficiency of the **Axum** framework.
- **AI Reasoning (Gemini 2.5 Flash):** 2.8s to 3.5s, which accounts for the majority of the wait time.
- **Regex Extraction & Formatting:** < 100ms, proving that post-processing JSON does not significantly impact user experience.

7.2.2 System Throughput

The system was stress-tested using simulated concurrent users. Due to the asynchronous nature of **FastAPI** and the lightweight threading of **Rust**, the gateway handled up to 100 concurrent requests without a significant spike in memory usage, remaining well within the **8GB RAM** limit of the experimental set-up.

7.3 DevOps and Orchestration Outcomes

The deployment phase provided insights into the stability of the **Kubernetes** and **Docker** infrastructure.

7.3.1 Self-Healing and Reliability

During stress tests, one instance of the AI Backend was intentionally terminated.

- **Outcome:** The Kubernetes **Liveness Probe** detected the failure within 10 seconds.
- **Recovery:** A new pod was automatically instantiated, and the **Rust Gateway** successfully redirected traffic to the healthy node, achieving 99.9% availability during the test window.

7.3.2 CI/CD Efficiency

The **GitHub Actions** pipeline was evaluated for deployment speed.

- **Build Time:** The multi-stage Docker build for the Rust gateway took approximately 4 minutes due to compilation overhead.
- **Deployment Time:** The rolling update to the **Minikube** cluster was completed in 45 seconds, ensuring that the new AI model parameters were updated with zero downtime.

7.4 Discussions and Key Findings

The investigation led to several significant technical conclusions:

- **The "Rust Advantage":** Using Rust for the gateway provided superior memory safety and stability compared to traditional Node.js proxies, especially when handling long-lived AI stream connections.

- **The Necessity of Regex:** It was found that without **Regex-based extraction**, 15% of Gemini's responses were unparsable by the frontend due to unnecessary conversational text.

- **Multimodal Impact:** Qualitative feedback indicated that users found the **Unsplash** medical imagery highly helpful in contextualizing the "cold" data of a treatment plan.

- **Scalability:** The microservice approach proved essential; scaling only the "Intelligence Layer" pods allowed the system to handle more queries without wasting resources on the frontend or gateway layers.

Chapter 8:

Summary and Conclusions

8.1 Summary of Work

The project successfully culminated in the design, development, and deployment of the **AI Healthcare Insurance Planner**, a sophisticated 3-tier microservices application. The primary objective was to demonstrate how **Agentic AI** and high-performance systems programming could be integrated to solve the problem of information asymmetry in medical and insurance planning.

Key achievements of the work include:

- **Architectural Innovation:** Implementation of a decoupled system using React.js for the frontend, Rust (Axum) for a high-concurrency gateway, and Python (FastAPI) for the AI logic.

- **Intelligence Layer:** Development of a multi-agent system using LangChain and Google Gemini 2.5 Flash, capable of generating structured treatment plans, INR cost estimations, and recovery schedules.

- **Data Integrity:** Deployment of Regex-based extraction logic that successfully bridged the gap between conversational LLM outputs and strict JSON requirements for the UI.

- **Visual Augmentation:** Implementation of a prompt-hashing algorithm that dynamically fetched professional medical imagery from Unsplash, providing a multimodal experience.

- **DevOps Excellence:** Full containerization via Docker and orchestration through Kubernetes, ensuring a self-healing and scalable production-ready environment.

8.2 Conclusions

Based on the results obtained during the testing and discussion phases, several critical conclusions can be drawn:

8.2.1 Effectiveness of Agentic AI

The transition from a single-prompt LLM to a **Multi-Agent System (MAS)** significantly enhanced the reliability of the system. By assigning specific roles to agents (Planner, Researcher, Formatter), the AI demonstrated a higher degree of accuracy in distinguishing between medical procedures and financial policy limitations.

8.2.2 Performance Benefits of the Rust Gateway

The use of **Rust** and the **Axum** framework proved to be a superior choice for the middleware layer. The gateway maintained a near-zero latency overhead and provided a memory-safe environment to handle CORS and reverse-proxying, which is often a bottleneck in Node.js-based architectures.

8.2.3 Criticality of Output Sanitization

The investigation confirmed that LLMs, including Gemini 2.5 Flash, are prone to including extra-textual information. The **Regex-based JSON extraction** was not merely an optimization but a structural necessity to prevent frontend parsing failures and ensure the stability of the glassmorphism dashboard.

8.2.4 Advantages of Microservice Orchestration

The **Kubernetes** deployment strategy validated that the system could remain highly available. The use of **Liveness and Readiness probes** proved effective in automated recovery, while the **Horizontal Pod Autoscaler (HPA)** demonstrated a scalable path for handling variable user loads without manual intervention.

8.3 Concluding Remarks

The **AI Healthcare Insurance Planner** represents a successful fusion of modern AI reasoning and robust infrastructure engineering.

- It provides a blueprint for future HealthTech applications, proving that complex healthcare financial planning can be made accessible, transparent, and visually engaging through a well-engineered microservices ecosystem.

8.4 Project Impact

The development of the AI Healthcare Insurance Planner marks a significant step toward solving the chronic issue of information asymmetry in the HealthTech domain. The successful integration of specialized **Agentic AI** with high-performance **DevOps** practices creates a highly resilient and adaptable system. This project establishes a new standard for delivering personalized financial and medical advice by validating the use of the **Rust (Axum) gateway** for memory-safe concurrency and the **Python (FastAPI)** backend for complex LLM orchestration. Beyond its technical achievements—such as the zero-downtime rolling updates achieved through **Kubernetes** and the strict data integrity ensured by the **Regex-based JSON Firewall**—the project's true impact lies in its potential to empower end-users. By localizing cost estimations in INR and simplifying complex insurance jargon through structured, visual dashboards, the application provides tangible support for crucial life decisions. Looking forward, the architecture is designed to accommodate significant growth. The foundation is set for advanced expansion into multi-language support, integration with real-time medical reports, and the establishment of robust regulatory compliance. This project is not merely a successful proof of concept, but a robust platform ready for its next phase of evolution, as detailed in the subsequent chapter on Future Scope.

Chapter 9:
Future Scope

9.1 Multi-Language and Vernacular Support

To truly democratize healthcare financial literacy across the Indian subcontinent, the next phase of the project involves implementing **Multilingual Support**. While the current system operates in English, integrating translation layers or fine-tuning models on vernacular datasets will allow users to query the system in Hindi, Marathi, Tamil, and other regional languages. This expansion will ensure that the **AI Healthcare Insurance Planner** remains accessible to a broader demographic, particularly in rural and semi-urban areas.

9.2 Integration of Real-Time Medical Reports

The future roadmap includes the transition from text-only prompts to **Multimodal Input Analysis**. By leveraging the native vision capabilities of **Google Gemini 2.5 Flash**, the system could allow users to upload physical medical reports, prescriptions, or hospital bills.

- **Automated Summarization:** The AI could extract key medical codes and diagnoses from uploaded images.

- **Comparison Logic:** The system could automatically compare a patient's actual hospital bill against the initial AI-generated INR estimate to identify discrepancies.

9.3 Regulatory Compliance and Data Privacy

As the system moves closer to a public production environment, integrating formal regulatory frameworks is a high-priority objective.

- **IRDAI Guidelines:** Future iterations will focus on aligning insurance recommendations with the standards set by the **Insurance Regulatory and Development Authority of India (IRDAI)**.
- **Health Data Privacy:** Implementing strict **HIPAA** or **DISHA** (Digital Information Security in Healthcare Act) compliance measures within the Kubernetes cluster will be necessary to protect sensitive user health data.

9.4 Advanced Agent Specialization

While the current multi-agent system uses a Planner, Researcher, and Formatter, future scope involves adding more "Specialist" agents:

- **Legal Agent:** To parse and explain "Fine Print" exclusions in insurance policies that are often missed by general researchers.
- **Recommendation Agent:** To suggest specific insurance products in the Indian market based on the user's chronic health history and financial profile.

9.5 Blockchain for Claim Transparency

There is a significant opportunity to integrate **Blockchain technology** for decentralized claim tracking. By storing AI-generated cost estimations on a ledger, the system could provide an immutable record that patients can use during claim disputes with insurance providers, further reducing the information gap.

9.6 Enhanced Observability and AIOps Implementation

The current DevOps pipeline focuses on core stability and deployment automation. The next strategic phase is to implement a robust **Observability Framework** that transitions the operations workflow from reactive monitoring to proactive, intelligent operations (AIOps). This is critical for maintaining service reliability and managing the non-deterministic nature of AI workloads in a production environment.

9.6.1 Real-Time Metrics and Monitoring (Prometheus & Grafana)

The existing **Kubernetes** environment will be instrumented with the Prometheus monitoring system. This involves deploying a dedicated Prometheus server and configuring service monitors to scrape metrics from all three microservices (React, Rust Gateway, FastAPI Backend).

- **Rust Gateway Metrics:** Focus on tracking connection lifetime, request queue depth, and I/O rates.

- **AI Backend Metrics:** Critical metrics include LLM token generation rate, LangChain agent step execution time, and success rates of the **Regex JSON extraction** logic.

- **Visualization:** Grafana dashboards will be created to provide real-time visualization of these metrics, allowing the operations team to immediately detect latency spikes or memory usage anomalies before they impact end-users.

9.6.2 Structured Logging and Distributed Tracing

To facilitate debugging across the polyglot microservices (Rust, Python), a unified logging solution is required.

- **Logging Stack:** Implementation of the ELK stack (Elasticsearch, Logstash, Kibana) or a modern alternative like Fluentd will standardize log formats.

- **Trace Context Propagation:** Every user request must be tagged with a unique trace ID at the React frontend. This ID will be propagated through the Rust Gateway and the Python AI Backend, enabling developers to trace the entire request-response lifecycle and pinpoint exactly where a bottleneck or failure occurred (Distributed Tracing).

9.6.3 AIOps for Automated Anomaly Detection

Moving beyond simple threshold alerts, the final stage is incorporating AIOps capabilities to manage AI-specific risks.

- **LLM Drift Detection:** Implementing models that monitor the statistical properties of the Gemini 2.5 Flash output (e.g., changes in average response length or semantic similarity) to detect performance degradation or "hallucination drift" over time.

- **Automated Scaling Logic:** Utilizing data from Prometheus and Grafana to feed machine learning models that predict future resource demand, allowing the **Horizontal Pod Autoscaler (HPA)** to scale pods up or down predictively, rather than reactively, optimizing cloud compute costs.

9.7 Predictive Modeling and Personalized Health Risk Assessment

Building upon the current system's ability to provide planning advice, the future scope will pivot toward **Proactive Health Risk Assessment** by integrating advanced statistical and machine learning models. This transformation shifts the AI from being a static planner to a dynamic, predictive health companion.

9.7.1 Health Risk Scoring and Stratification

The system will be expanded to take a broader range of user input (e.g., family history, blood pressure, lifestyle habits) to calculate a personalized **Health Risk Score**.

- **Machine Learning Models:** Deployment of specialized ML models (e.g., Random Forests or Gradient Boosting Machines) trained on anonymized public health datasets to assess the long-term risk of developing common chronic diseases (e.g., hypertension, heart disease, certain cancers).

- **Risk Stratification:** Users will be categorized into risk tiers (Low, Medium, High). This stratification serves as the input for the **Recommendation Agent** to tailor advice.

9.7.2 Advanced Personalization of Insurance Products

The current **Recommendation Agent** suggests broad policy categories. With the integration of predictive models, the recommendations will become significantly more granular.

- **Predictive Policy Matching:** The AI will match the user’s specific health risk score and predicted future medical expenditure against thousands of policy riders and exclusions from various Indian insurers.

- **Optimized Coverage Advice:** Instead of general recommendations, the system will advise on optimal Sum Insured values, suggest adding riders for specific predicted risks (e.g., critical illness riders for high-risk users), and calculate the long-term premium-to-benefit ratio.

9.7.3 Ethical AI and Privacy in Predictive Systems

The implementation of predictive modeling necessitates rigorous adherence to ethical AI principles, particularly regarding sensitive health data.

- **Explainable AI (XAI):** Implementing techniques (like SHAP values) to explain *why* a user received a specific health risk score or policy recommendation. This builds user trust and complies with future regulatory demands for transparency.

- **Privacy-Preserving Techniques:** Investigating the use of **Federated Learning** to train predictive models without ever centralizing user-specific health data, ensuring maximum data privacy while maintaining high model accuracy.

Appendix

Appendix A: Technical Implementation (Core Snippets)

A.1 Middleware Gateway (Rust / Axum)

The following snippet illustrates the high-performance reverse-proxy logic implemented in Rust to route requests to the AI backend while managing CORS.

```
use axum::{routing::post, Router, Json};
use tower_http::cors::CorsLayer;

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/api/gateway/query", post(proxy_to_ai))
        .layer(CorsLayer::permissive()); // Defined for development flexibility

    axum::Server::bind(&"0.0.0.0:8080".parse().unwrap())
        .serve(app.into_make_service())
        .await
        .unwrap();
}
```

A.2 Output Sanitization (Python / Regex)

This utility is used within the FastAPI backend to ensure that the Gemini 2.5 Flash output is a strictly parsable JSON object, removing any conversational "noise".

```
import re
import json
def extract_clean_json(llm_output):
    # Regex to find content between the first and last curly braces
    match = re.search(r'\{.*\}', llm_output, re.DOTALL)
    if match:
        return json.loads(match.group())
    return {"error": "Invalid JSON format from LLM"}
```

Appendix B: DevOps and Infrastructure Manifests

B.1 Multi-Stage Dockerfile (AI Backend)

Implemented to minimize the attack surface and optimize deployment speed in Kubernetes.

```
# Stage 1: Build
FROM python:3.10-slim as builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: Runtime
FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

B.2 Kubernetes Deployment Manifest (K8s)

Standardized deployment template including Liveness and Readiness Probes for system self-healing.

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-planner
  template:
    metadata:
      labels:
        app: ai-planner
    spec:
      containers:
      - name: fastapi-server
        image: btechgroup07/ai-backend:latest
        ports:
        - containerPort: 8000
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 10
```

Bibliography

[1] M. Wooldridge and N. R. Jennings, "Intelligent Agents: Theory and Practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115-152, 1995.

[2] K. P. Sycara, "Multiagent systems," *AI Magazine*, vol. 19, no. 2, p. 79, 1998.

[3] L. Chen et al., "Kubernetes-based Deployment of Large Language Model Applications," *IEEE Transactions on Cloud Computing*, vol. 12, no. 1, pp. 45-59, 2024.

[4] K. Bala et al., "MLOps: Practices and Challenges in Deploying Machine Learning Models," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1-35, 2023.

[5] Google AI Studio, "Gemini 2.5 Flash Model Documentation and API Reference," 2025. [Online]. Available: https://ai.google.dev/docs.

[6] LangChain, "Building Agentic Workflows and Multi-Agent Systems," 2025. [Online]. Available: https://python.langchain.com/docs.

[7] Axum Framework, "Ergonomic and Modular Web Framework for Rust," 2025. [Online]. Available: https://docs.rs/axum.

[8] FastAPI Documentation, "Modern, High-Performance Python Web Framework," 2025. [Online]. Available: https://fastapi.tiangolo.com.

[9] Docker, Inc., "Multi-stage Build Strategies and Container Best Practices," 2025. [Online]. Available: https://docs.docker.com.

[10] Kubernetes Authors, "Orchestrating Microservices with Kubernetes: Liveness and Readiness Probes," 2025. [Online]. Available: https://kubernetes.io/docs.

[11] Vite.js, "Next Generation Frontend Tooling for React," 2025. [Online]. Available: https://vitejs.dev/guide.

[12] IRDAI, "Guidelines on Standard Insurance Product Disclosure and Digital Literacy," Insurance Regulatory and Development Authority of India, 2024.